Meeting 7 — System Timing Specification

Phase: 3.1 — Runtime Architecture Design
Meeting Objective: Define the execution frequency, timing requirements, deadlines, and real-time classification of every subsystem in the Digital Harmonium Workstation.

---

1. Purpose

The previous meeting defined the runtime architecture and execution layers.

This meeting answers a different question:

> **Exactly when does every subsystem execute?**

A digital musical instrument is fundamentally a real-time system.

Every subsystem has its own timing requirements.

Some operations must execute every audio block.

Others only when a button is pressed.

Others can execute once every second.

Understanding these timing requirements is essential for designing a deterministic firmware.

---

2. Timing Philosophy

The workstation should never execute software "whenever possible."

Instead, every subsystem belongs to one of four timing classes:

Class A — Hard Real-Time

Failure causes audible glitches.

Examples:

- DMA
- Audio rendering

- Voice processing
- Mixer

Class B — Soft Real-Time

Should remain responsive but minor delays are acceptable.

Examples:

- Keyboard scanning
- Button scanning
- LED updates
- Metronome scheduling

Class C — Human-Time

Limited by human perception rather than processor speed.

Examples:

- LCD updates
- Menu navigation
- Screen refresh
- Status messages

Class D — Background

Runs only when CPU time is available.

Examples:

- SD card writes
- Diagnostics
- USB communication
- Statistics
- Configuration saving

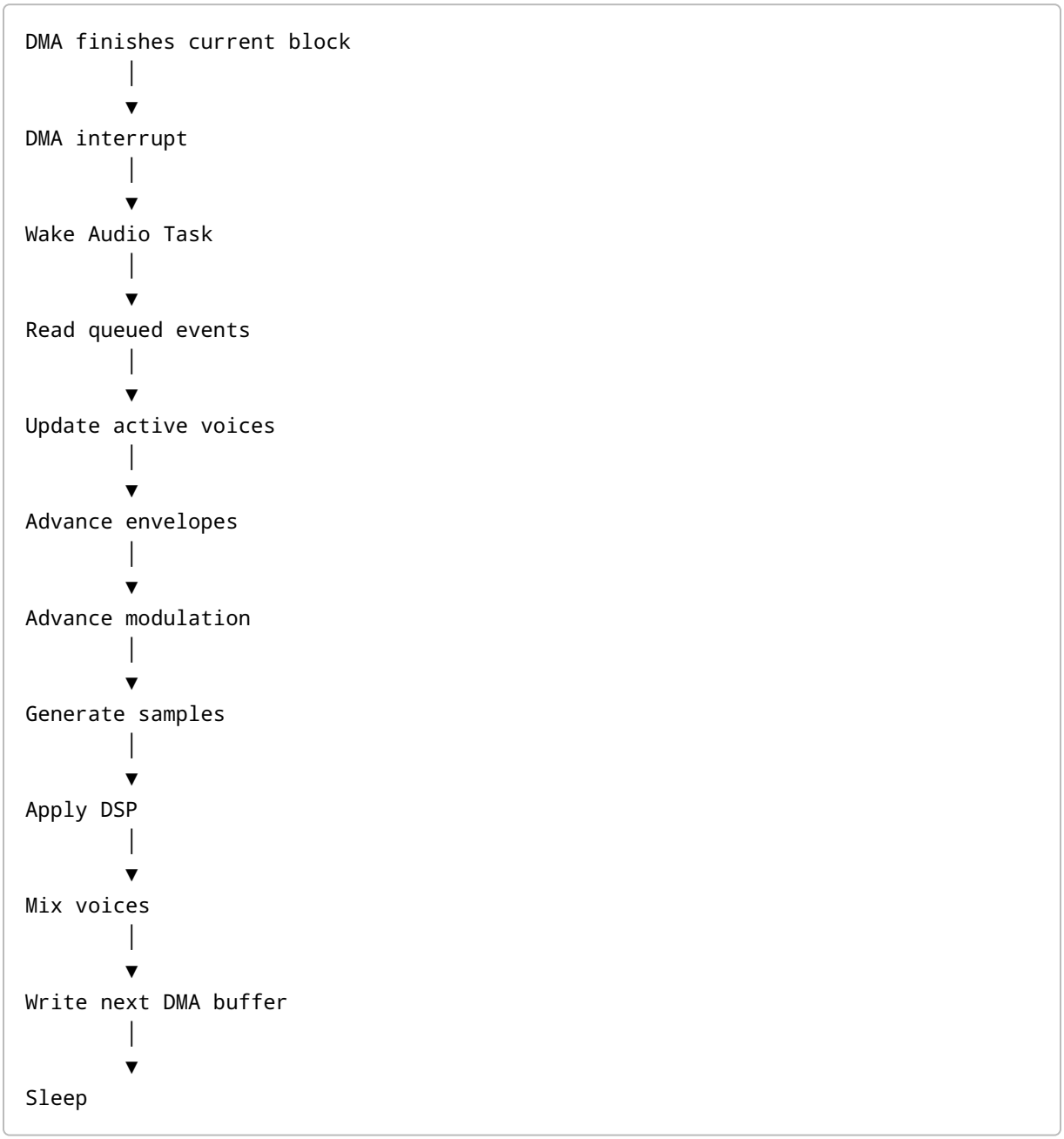
---

3. Master Timing Table

Subsystem	Trigger	Frequency	Timing Class
DMA Engine	Hardware	Every audio block	Hard
Audio Task	DMA notification	Every 11.6 ms	Hard
Voice Manager	Audio Task	Every block	Hard
Envelope Engine	Audio Task	Every block	Hard
LFO Engine	Audio Task	Every block	Hard
Pitch Engine	Audio Task	Every block	Hard
Sample Engine	Audio Task	Every block	Hard
Filter Engine	Audio Task	Every block	Hard
Mixer	Audio Task	Every block	Hard
Master Output	Audio Task	Every block	Hard
Keyboard Scanner	Periodic	1 ms	Soft
Button Scanner	Periodic	2 ms	Soft
LED Manager	Periodic	100–200 Hz	Soft
Tabla Engine	Audio Task	Every block	Hard
Tanpura Engine	Audio Task	Every block	Hard
Metronome Engine	Internal timer	Beat dependent	Soft
Recorder	Audio Task	Every block	Hard
Playback Engine	Audio Task	Every block	Hard
LCD Manager	Periodic	20–30 FPS	Human
Menu Manager	User events	On demand	Human
Instrument Loader	User request	Event driven	Background
SD Writer	Event	On demand	Background
USB Manager	Connection	Event driven	Background
Diagnostics	Periodic	500 ms–1 s	Background
Statistics	Periodic	1 s	Background

4. Audio Timeline

Every audio block follows exactly the same schedule.



Target execution time:

Less than the duration of one audio block.

---

5. Keyboard Timing

Keyboard response determines how responsive the instrument feels.

Target scan frequency:

1 millisecond

This ensures:

- Very low key latency
- Fast chord detection
- Reliable velocity estimation
- Responsive performance

The keyboard scanner only detects changes.

Actual sound generation occurs later inside the Audio Runtime.

---

6. Button Timing

Buttons do not require the same speed as keyboard keys.

Recommended scan interval:

2 milliseconds

Benefits:

- Eliminates bounce filtering delays
- Responsive menus
- Low CPU usage

---

7. LCD Timing

Humans cannot perceive extremely fast LCD updates.

Target refresh:

20–30 frames per second

This provides:

- Smooth navigation
- Stable text
- Minimal CPU consumption

The LCD must never update directly from the Audio Task.

---

8. LED Timing

Keyboard LEDs should appear instantaneous.

Recommended refresh:

100–200 Hz

This avoids visible flicker while leaving sufficient CPU time for audio generation.

---

9. Recording Timing

Recording is synchronized with the Audio Runtime.

Each audio block:

- Reads newly generated samples
- Stores recording data
- Updates recording position

No separate recording clock is required.

---

10. Playback Timing

Playback uses the same timing as recording.

Each block:

- Reads recorded events
- Injects playback commands
- Updates LED indicators
- Advances playback timer

Synchronization is maintained automatically because playback shares the Audio Runtime.

---

11. Tabla Timing

Tabla playback is event driven.

The engine determines:

- Current beat
- Current subdivision
- Next bol
- Sample scheduling

Actual sample rendering still occurs inside the Audio Runtime.

---

12. Tanpura Timing

The tanpura engine continuously schedules the drone pattern.

Responsibilities include:

- String sequence progression
- Timing between plucks
- Loop advancement
- Synchronization with transpose

Like the tabla engine, it schedules events rather than generating audio independently.

---

13. Metronome Timing

The metronome maintains a musical clock.

Responsibilities:

- BPM calculation
- Beat counting
- Accent placement
- Tick scheduling

The metronome should remain synchronized with recording and playback.

---

14. Background Timing

Background modules execute only when higher-priority tasks are idle.

Typical execution intervals:

Module	Typical Interval
Diagnostics	500 ms
Runtime Statistics	1 s
Memory Monitor	1 s
USB Polling	Event driven
SD Operations	On demand

Background timing is intentionally flexible.

---

15. CPU Budget Philosophy

The CPU should always reserve sufficient time for unexpected workload increases.

Conceptual allocation:

Layer	Approximate Budget
Audio Runtime	~70%
Control Runtime	~15%
Display Runtime	~5%
Background Runtime	~10%

These values are design goals rather than fixed measurements.

---

16. Deadline Rules

Every subsystem has an implicit deadline.

Hard Deadline

Must complete before the next audio block begins.

Examples:

- Audio rendering
- Mixing
- Voice processing

Failure results in audible glitches.

Soft Deadline

Completion may be delayed slightly without affecting audio quality.

Examples:

- Button handling
- LED updates

Human Deadline

Completion within tens of milliseconds is acceptable.

Examples:

- LCD refresh
- Menu navigation

Background Deadline

No strict deadline.

Execution occurs only when processor time is available.

---

17. Timing Rules

The firmware must always obey these rules:

1. Audio timing always has absolute priority.
2. No filesystem operations inside the Audio Runtime.
3. No display updates inside interrupt routines.
4. No dynamic memory allocation during audio rendering.
5. Keyboard scanning must never block the Audio Runtime.
6. Background work may be delayed indefinitely.
7. DMA timing is the master clock for the entire system.
8. Every subsystem should have a clearly defined execution frequency.
9. CPU idle time is desirable and should be preserved.
10. Deterministic execution is always preferred over maximum CPU utilization.

---

18. Final Timing Philosophy

The Digital Harmonium Workstation behaves like a musical instrument rather than a conventional embedded application.

Its heartbeat is the audio clock.

Every subsystem executes according to its own timing requirements, but all are ultimately synchronized around the continuous production of audio.

The result is a firmware architecture that is deterministic, responsive, low-latency, and scalable as additional instruments and features are introduced.